# Prolog

Set and Bag

# setof and bagof

- These predicates collect all solutions to a goal

```
bagof(Template, Goal, Bag)
setof(Template, Goal, Set)
```

- They backtracks over free variables, producing one bag/set per binding
  - They treat unbound variables in the Goal as grouping keys
  - The existential quantifiers ^ is used when you don't want a variable to act as a grouping key

# The setof predicate

```
like(jean,apple).
like(marie,pear).

?- setof([X,Y],like(X,Y),L).
L=[[jean,apple],[marie,pear]].
```

```
age(pierre,5).
age(paul,7).
age(henri,5).

?- setof(C,age(C,5),L).
L=[henri,pierre].
```

*bagof is similar except that it does not eliminate duplicate and does not sort the element*

# Example

```
bag(2,4,1).          % bagof(Z,bag(X,Y,Z),B).
bag(3,5,2).          % bagof(Z,(bag(X,Y,Z),Z>2),B).
bag(7,8,2).          % bagof(Z,X^bag(X,Y,Z),B).
bag(4,3,1).          % setof(Z,X^bag(X,Y,Z),B).
bag(5,2,4).          % bagof(Z,X^Y^bag(X,Y,Z),B).
bag(2,1,4).          % findall(Z,bag(X,Y,Z),B).
bag(2,2,4).
bag(7,3,5).
bag(7,3,3).
```

# Example

```
knows(vincent,david).
knows(vincent,antoine).
knows(vincent,alex).
knows(melodie,alex).
knows(melodie,patrick).
knows(patrick,melodie).
knows(patrick,ahmed).
knows(patrick,eddie).
knows(patrick,david).
```

```
% setof(X,knows(X,Y),B).
% setof(Y,knows(X,Y),B).
% setof(Y,X^knows(X,Y),B).
% bagof(Y,X^knows(X,Y),B).
% setof([X,Y],knows(X,Y),B).
```

# Example

```
age(vincent,8).
age(melodie,4).
age(patrick,3).
age(ahmed,7).
age(eddie,4).
% setof(A,N^age(N,A),B).
% setof(A,N^age(N,A),[T|Q]).
% setof(A,N^age(N,A),[T|_]).
% setof([A,N],age(N,A),[[_,J]|_]).
% age(P,A1),\+((age(_,A2),A2<A1)).
```

# The farmer riddle

A man has to get a fox, a chicken, and a sack of corn across a river. He has a rowboat, and it can only carry him and one other thing. If the fox and the chicken are left together, the fox will eat the chicken. If the chicken and the corn are left together, the chicken will eat the corn.

```
% state(farmer,fox,chicken,corn).

initial(state([left, left, left, left])).
final(state([right, right, right, right])).
```

# The farmer riddle

```
traverse(state([left,X,Y,Z]),state([right,X,Y,Z]), farmer_crosses).
traverse(state([right,X,Y,Z]),state([left,X,Y,Z]), farmer_comes_back).

traverse(state([left,X,left,Z]),state([right,X,right,Z]), farmer_chicken_cross).
traverse(state([right,X,right,Z]),state([left,X,left,Z]), farmer_chicken_come_back).

traverse(state([left, left, X, Y]),state([right, right, X, Y]), farmer_fox_cross).
traverse(state([right, right, X, Y]),state([left, left, X, Y]), farmer_fox _come_back).

traverse(state([left, X, Y, left]),state([right, X, Y, right]), farmer_corn_cross).
traverse(state([right, X, Y, right]),state([left, X, Y, left]), farmer_corn _come_back).
```

# The farmer riddle

```
forbidden(state([X, Y, Y, _])) :- X \== Y.
forbidden(state([X, _, Y, Y])) :- X \== Y.

river(P) :-
    initial(Start), final(End),
    river_aux(Start, End, [Start], P).
```

# The farmer riddle

```
river_aux(A,A,_,[]).

% V contains the visited states
river_aux(A, B,V, P) :-
    traverse(A,C,Action),
    not(forbidden(C)),
    not(member(C,V)),
    river_aux(C,B,[C|V],Plan),
    P = [Action | Plan].
```